





Build a Large Language Model (From Scratch)

Sebastian Raschka

To comment go to [livebook](#).



For more information on this and other Manning titles go to [manning.com](#).

copyright

For online information and ordering of this and other Manning books, please visit [www.manning.com](#). The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department

Manning Publications Co.

20 Baldwin Road

PO Box 761

Shelter Island, NY 11964

Email: orders@manning.com

©2025 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

☑ Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The authors and publisher have made every effort to ensure that the information in this book was correct at press time. The authors and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor: Dustin Archibald
Technical editor: David Caswell
Review editor: Kishor Rit
Production editor: Aleksandar Dragosavljević
Copy editors: Kari Lucke and Alisa Larson
Proofreader: Mike Beady
Technical proofreader: Jerry Kuch
Typesetter: Dennis Dalinnik
Cover designer: Marija Tudor

ISBN: 9781633437166

Printed in the United States of America

contents

[preface](#)

[acknowledgments](#)

[about this book](#)

[about the author](#)

[about the cover illustration](#)

[1 Understanding large language models](#)

[1.1 What is an LLM?](#)

[1.2 Applications of LLMs](#)

[1.3 Stages of building and using LLMs](#)

[1.4 Introducing the transformer architecture](#)

[1.5 Utilizing large datasets](#)

[1.6 A closer look at the GPT architecture](#)

[1.7 Building a large language model](#)

[2 Working with text data](#)

[2.1 Understanding word embeddings](#)

[2.2 Tokenizing text](#)

[2.3 Converting tokens into token IDs](#)

[2.4 Adding special context tokens](#)

[2.5 Byte pair encoding](#)

[2.6 Data sampling with a sliding window](#)

[2.7 Creating token embeddings](#)

[2.8 Encoding word positions](#)

[3 Coding attention mechanisms](#)

[3.1 The problem with modeling long sequences](#)

[3.2 Capturing data dependencies with attention mechanisms](#)

[3.3 Attending to different parts of the input with self-attention](#)

[3.3.1 A simple self-attention mechanism without trainable weights](#)

[3.3.2 Computing attention weights for all input tokens](#)

[3.4 Implementing self-attention with trainable weights](#)

[3.4.1 Computing the attention weights step by step](#)

[3.4.2 Implementing a compact self-attention Python class](#)

[3.5 Hiding future words with causal attention](#)

[3.5.1 Applying a causal attention mask](#)

[3.5.2 Masking additional attention weights with dropout](#)

[3.5.3 Implementing a compact causal attention class](#)

[3.6 Extending single-head attention to multi-head attention](#)

[3.6.1 Stacking multiple single-head attention layers](#)

[3.6.2 Implementing multi-head attention with weight splits](#)

[4 Implementing a GPT model from scratch to generate text](#)

[4.1 Coding an LLM architecture](#)

[4.2 Normalizing activations with layer normalization](#)

[4.3 Implementing a feed forward network with GELU activations](#)

[4.4 Adding shortcut connections](#)

[4.5 Connecting attention and linear layers in a transformer block](#)

[4.6 Coding the GPT model](#)

[4.7 Generating text](#)

[5 Pretraining on unlabeled data](#)

[5.1 Evaluating generative text models](#)

[5.1.1 Using GPT to generate text](#)

[5.1.2 Calculating the text generation loss](#)

[5.1.3 Calculating the training and validation set losses](#)

[5.2 Training an LLM](#)

[5.3 Decoding strategies to control randomness](#)

[5.3.1 Temperature scaling](#)

[5.3.2 Top-k sampling](#)

[5.3.3 Modifying the text generation function](#)

[5.4 Loading and saving model weights in PyTorch](#)

[5.5 Loading pretrained weights from OpenAI](#)

6 Fine-tuning for classification

- [6.1 Different categories of fine-tuning](#)
- [6.2 Preparing the dataset](#)
- [6.3 Creating data loaders](#)
- [6.4 Initializing a model with pretrained weights](#)
- [6.5 Adding a classification head](#)
- [6.6 Calculating the classification loss and accuracy](#)
- [6.7 Fine-tuning the model on supervised data](#)
- [6.8 Using the LLM as a spam classifier](#)

7 Fine-tuning to follow instructions

- [7.1 Introduction to instruction fine-tuning](#)
- [7.2 Preparing a dataset for supervised instruction fine-tuning](#)
- [7.3 Organizing data into training batches](#)
- [7.4 Creating data loaders for an instruction dataset](#)
- [7.5 Loading a pretrained LLM](#)
- [7.6 Fine-tuning the LLM on instruction data](#)
- [7.7 Extracting and saving responses](#)
- [7.8 Evaluating the fine-tuned LLM](#)
- [7.9 Conclusions](#)
 - [7.9.1 What's next?](#)
 - [7.9.2 Staying up to date in a fast-moving field](#)
 - [7.9.3 Final words](#)

appendix A Introduction to PyTorch

- [A.1 What is PyTorch?](#)
 - [A.1.1 The three core components of PyTorch](#)
 - [A.1.2 Defining deep learning](#)
 - [A.1.3 Installing PyTorch](#)
- [A.2 Understanding tensors](#)
 - [A.2.1 Scalars, vectors, matrices, and tensors](#)

[A.2.2 Tensor data types](#)

[A.2.3 Common PyTorch tensor operations](#)

[A.3 Seeing models as computation graphs](#)

[A.4 Automatic differentiation made easy](#)

[A.5 Implementing multilayer neural networks](#)

[A.6 Setting up efficient data loaders](#)

[A.7 A typical training loop](#)

[A.8 Saving and loading models](#)

[A.9 Optimizing training performance with GPUs](#)

[A.9.1 PyTorch computations on GPU devices](#)

[A.9.2 Single-GPU training](#)

[A.9.3 Training with multiple GPUs](#)

[appendix B References and further reading](#)

[appendix C Exercise solutions](#)

[appendix D Adding bells and whistles to the training loop](#)

[D.1 Learning rate warmup](#)

[D.2 Cosine decay](#)

[D.3 Gradient clipping](#)

[D.4 The modified training function](#)

[appendix E Parameter-efficient fine-tuning with LoRA](#)

[E.1 Introduction to LoRA](#)

[E.2 Preparing the dataset](#)

[E.3 Initializing the model](#)

[E.4 Parameter-efficient fine-tuning with LoRA](#)

[index](#)

preface

I've always been fascinated with language models. More than a decade ago, my journey into AI began with a statistical pattern classification class, which led to

my first independent project: developing a model and web application to detect the mood of a song based on its lyrics.

Fast forward to 2022, with the release of ChatGPT, large language models (LLMs) have taken the world by storm and have revolutionized how many of us work. These models are incredibly versatile, aiding in tasks such as checking grammar, composing emails, summarizing lengthy documents, and much more. This is owed to their ability to parse and generate human-like text, which is important in various fields, from customer service to content creation, and even in more technical domains like coding and data analysis.

As their name implies, LLMs are “large”—very large—encompassing millions to billions of parameters. (For comparison, using more traditional machine learning or statistical methods, the Iris flower dataset can be classified with more than 90% accuracy using a small model with only two parameters.) However, despite the large size of LLMs compared to more traditional methods, LLMs don’t have to be a black box.

In this book, you will learn how to build an LLM one step at a time. By the end, you will have a solid understanding of how an LLM, like the ones used in ChatGPT, works on a fundamental level. I believe that developing confidence with each part of the fundamental concepts and underlying code is crucial for success. This not only helps in fixing bugs and improving performance but also enables experimentation with new ideas.

Several years ago, when I started working with LLMs, I had to learn how to implement them the hard way, sifting through many research papers and incomplete code repositories to develop a general understanding. With this book, I hope to make LLMs more accessible by developing and sharing a step-by-step implementation tutorial detailing all the major components and development phases of an LLM.

I strongly believe that the best way to understand LLMs is to code one from scratch—and you’ll see that this can be fun too!

Happy reading and coding!

acknowledgments

Writing a book is a significant undertaking, and I would like to express my sincere gratitude to my wife, Liza, for her patience and support throughout this process. Her unconditional love and constant encouragement have been absolutely essential.

I am incredibly grateful to Daniel Kleine, whose invaluable feedback on the in-progress chapters and code went above and beyond. With his keen eye for

detail and insightful suggestions, Daniel's contributions have undoubtedly made this book a smoother and more enjoyable reading experience.

I would also like to thank the wonderful staff at Manning Publications, including Michael Stephens, for the many productive discussions that helped shape the direction of this book, and Dustin Archibald, whose constructive feedback and guidance in adhering to the Manning guidelines have been crucial. I also appreciate your flexibility in accommodating the unique requirements of this unconventional from-scratch approach. A special thanks to Aleksandar Dragosavljević, Kari Lucke, and Mike Beady for their work on the professional layouts and to Susan Honeywell and her team for refining and polishing the graphics.

I want to express my heartfelt gratitude to Robin Campbell and her outstanding marketing team for their invaluable support throughout the writing process.

Finally, I extend my thanks to the reviewers: Anandaganesh Balakrishnan, Anto Aravinth, Ayush Bihani, Bassam Ismail, Benjamin Muskalla, Bruno Sonnino, Christian Prokopp, Daniel Kleine, David Curran, Dibyendu Roy Chowdhury, Gary Pass, Georg Sommer, Giovanni Alzetta, Guillermo Alcántara, Jonathan Reeves, Kunal Ghosh, Nicolas Modrzyk, Paul Silisteanu, Raul Ciotescu, Scott Ling, Sriram Macharla, Sumit Pal, Vahid Mirjalili, Vaijanath Rao, and Walter Reade for their thorough feedback on the drafts. Your keen eyes and insightful comments have been essential in improving the quality of this book.

To everyone who has contributed to this journey, I am sincerely grateful. Your support, expertise, and dedication have been instrumental in bringing this book to fruition. Thank you!

about this book

Build a Large Language Model (From Scratch) was written to help you understand and create your own GPT-like large language models (LLMs) from the ground up. It begins by focusing on the fundamentals of working with text data and coding attention mechanisms and then guides you through implementing a complete GPT model from scratch. The book then covers the pretraining mechanism as well as fine-tuning for specific tasks such as text classification and following instructions. By the end of this book, you'll have a deep understanding of how LLMs work and the skills to build your own models. While the models you'll create are smaller in scale compared to the large foundational models, they use the same concepts and serve as powerful educational tools to grasp the core mechanisms and techniques used in building state-of-the-art LLMs.

Who should read this book

Build a Large Language Model (From Scratch) is for machine learning enthusiasts, engineers, researchers, students, and practitioners who want to gain a deep understanding of how LLMs work and learn to build their own models from scratch. Both beginners and experienced developers will be able to use their existing skills and knowledge to grasp the concepts and techniques used in creating LLMs.

What sets this book apart is its comprehensive coverage of the entire process of building LLMs, from working with datasets to implementing the model architecture, pretraining on unlabeled data, and fine-tuning for specific tasks. As of this writing, no other resource provides such a complete and hands-on approach to building LLMs from the ground up.

To understand the code examples in this book, you should have a solid grasp of Python programming. While some familiarity with machine learning, deep learning, and artificial intelligence can be beneficial, an extensive background in these areas is not required. LLMs are a unique subset of AI, so even if you're relatively new to the field, you'll be able to follow along.

If you have some experience with deep neural networks, you may find certain concepts more familiar, as LLMs are built upon these architectures. However, proficiency in PyTorch is not a prerequisite. Appendix A provides a concise introduction to PyTorch, equipping you with the necessary skills to comprehend the code examples throughout the book.

A high school-level understanding of mathematics, particularly working with vectors and matrices, can be helpful as we explore the inner workings of LLMs. Advanced mathematical knowledge is not necessary to grasp the key concepts and ideas presented in this book.

The most important prerequisite is a strong foundation in Python programming. With this knowledge, you'll be well prepared to explore the fascinating world of LLMs and understand the concepts and code examples presented in this book.

How this book is organized: A roadmap

This book is designed to be read sequentially, as each chapter builds upon the concepts and techniques introduced in the previous ones. The book is divided into seven chapters that cover the essential aspects of LLMs and their implementation.

Chapter 1 provides a high-level introduction to the fundamental concepts behind LLMs. It explores the transformer architecture, which forms the basis for LLMs such as those used on the ChatGPT platform.

Chapter 2 lays out a plan for building an LLM from scratch. It covers the process of preparing text for LLM training, including splitting text into word and subword tokens, using byte pair encoding for advanced tokenization, sampling training examples with a sliding window approach, and converting tokens into vectors that feed into the LLM.

Chapter 3 focuses on the attention mechanisms used in LLMs. It introduces a basic self-attention framework and progresses to an enhanced self-attention mechanism. The chapter also covers the implementation of a causal attention module that enables LLMs to generate one token at a time, masking randomly selected attention weights with dropout to reduce overfitting and stacking multiple causal attention modules into a multihead attention module.

Chapter 4 focuses on coding a GPT-like LLM that can be trained to generate human-like text. It covers techniques such as normalizing layer activations to stabilize neural network training, adding shortcut connections in deep neural networks to train models more effectively, implementing transformer blocks to create GPT models of various sizes, and computing the number of parameters and storage requirements of GPT models.

Chapter 5 implements the pretraining process of LLMs. It covers computing the training and validation set losses to assess the quality of LLM-generated text, implementing a training function and pretraining the LLM, saving and loading model weights to continue training an LLM, and loading pretrained weights from OpenAI.

Chapter 6 introduces different LLM fine-tuning approaches. It covers preparing a dataset for text classification, modifying a pretrained LLM for fine-tuning, fine-tuning an LLM to identify spam messages, and evaluating the accuracy of a fine-tuned LLM classifier.

Chapter 7 explores the instruction fine-tuning process of LLMs. It covers preparing a dataset for supervised instruction fine-tuning, organizing instruction data in training batches, loading a pretrained LLM and fine-tuning it to follow human instructions, extracting LLM-generated instruction responses for evaluation, and evaluating an instruction-fine-tuned LLM.

About the code

To make it as easy as possible to follow along, all code examples in this book are conveniently available on the Manning website at <https://www.manning.com/books/build-a-large-language-model-from-scratch>, as well as in Jupyter notebook format on GitHub at <https://github.com/rasbt/LLMs-from-scratch>. And don't worry about getting stuck—solutions to all the code exercises can be found in appendix C.

This book contains many examples of source code both in numbered listings and in line with normal text. In both cases, source code is formatted in a `fixed-width font like this` to separate it from ordinary text.

In many cases, the original source code has been reformatted; we've added line breaks and reworked indentation to accommodate the available page space in the book. In rare cases, even this was not enough, and listings include line-continuation markers (`\>`). Additionally, comments in the source code have often been removed from the listings when the code is described in the text. Code annotations accompany many of the listings, highlighting important concepts.

One of the key goals of this book is accessibility, so the code examples have been carefully designed to run efficiently on a regular laptop, without the need for any special hardware. But if you do have access to a GPU, certain sections provide helpful tips on scaling up the datasets and models to take advantage of that extra power.

Throughout the book, we'll be using PyTorch as our go-to tensor and a deep learning library to implement LLMs from the ground up. If PyTorch is new to you, I recommend you start with appendix A, which provides an in-depth introduction, complete with setup recommendations.

liveBook discussion forum

Purchase of *Build a Large Language Model (From Scratch)* includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to <https://livebook.manning.com/book/build-a-large-language-model-from-scratch/discussion>. You can also learn more about Manning's forums and the rules of conduct at <https://livebook.manning.com/discussion>.

Manning's commitment to readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the author can take place. It is not a commitment to any specific amount of participation on the part of the author, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking the author some challenging questions lest his interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.

Other online resources

Interested in the latest AI and LLM research trends?

- Check out my blog at <https://magazine.sebastianraschka.com>, where I regularly discusses the latest AI research with a focus on LLMs.

Need help getting up to speed with deep learning and PyTorch?

- I offer several free courses on my website at <https://sebastianraschka.com/teaching>. These resources can help you quickly get up to speed with the latest techniques.

Looking for bonus materials related to the book?

- Visit the book's GitHub repository at <https://github.com/rasbt/LLMs-from-scratch> to find additional resources and examples to supplement your learning.

about the author



Sebastian Raschka, PhD, has been working in machine learning and AI for more than a decade. In addition to being a researcher, Sebastian has a strong passion for education. He is known for his bestselling books on machine learning with Python and his contributions to open source.

Sebastian is a staff research engineer at Lightning AI, focusing on implementing and training LLMs. Before his industry experience, Sebastian was an assistant professor in the Department of Statistics at the University of Wisconsin-Madison, where he focused on deep learning research. You can learn more about Sebastian at <https://sebastianraschka.com>.

about the cover illustration

The figure on the cover of *Build a Large Language Model (From Scratch)*, titled "Le duchesse," or "The duchess," is taken from a book by Louis Curmer published in 1841. Each illustration is finely drawn and colored by hand.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

1 Understanding large language models

This chapter covers

- High-level explanations of the fundamental concepts behind large language models (LLMs)
- Insights into the transformer architecture from which LLMs are derived

This document was truncated here because it was created in the Evaluation Mode.

